TP : Cryptography in Action: An OpenSSL Lab

Imane Fouad, UM6P

Learning Objectives
By the end of this practical session, students will be able to:

1. Encrypt and decrypt a file using AES.
2. Understand symmetric encryption and shared keys.
3. Generate RSA public and private keys.
4. Compare the security properties of ECB and CBC modes.
5. Encrypt and decrypt a short message using RSA.

Your assignment

The CC@UM6P security team intercepts a new encrypted file. The attackers are confident their secrets are safe, but a hidden weakness may reveal more than expected.
Your mission: recover an RSA-protected password, uncover AES encryption weaknesses, unlock the secret file, and secure communications using RSA keys
Evidence: `classified.enc` **Deliverable:** completed investigation sheet.

Mission 1 — The RSA Password Challenge

MISSION BRIEFING — Access Denied!

A confidential document has been discovered on a university research server.
Unfortunately, the document is encrypted, and the password is unknown.
The security team intercepted an RSA-encrypted password during an earlier communication.
Your mission: Recover the original password using your knowledge of RSA cryptography.

Intercepted evidence:

$$\boxed{p = 5, \qquad q = 11, \qquad e = 3, \qquad C = 23}$$

You have all the information needed to recover the password. No guessing required!

Challenge 1.1 — Reconstruct the RSA System

Before decrypting the password, you must reconstruct the RSA parameters.

1. Determine the RSA public key (n, e) and private key (n, d).
2. Decrypt the intercepted ciphertext $C = 23$ to recover the original message m.
3. Verify your result by encrypting the recovered message again.
   What do you observe?

STUCK? Forgotten Your RSA Course? Here Is Your Rescue Kit!

Before reading the hints, make sure you have attempted the challenge yourself. A real cryptanalyst does not give up so easily!

Hint 1 — RSA Parameters
RSA uses two distinct prime numbers p and q.
The RSA modulus is:

$$n = pq$$

Euler's totient is:

$$\varphi(n) = (p-1)(q-1)$$

Hint 2 — The Private Exponent
The public exponent e and private exponent d satisfy:

$$ed \equiv 1 \pmod{\varphi(n)}$$

For small numbers, try successive positive values of d until the remainder is 1.

Hint 3 — RSA Decryption
The original message is recovered using:

$$m = C^d \bmod n$$

Hint 4 — Verify Your Answer
If the recovered message is correct, encrypting it again should reproduce the original ciphertext:

$$m^e \bmod n = C$$

Mission 2 — Break the Image, Not AES!

MISSION BRIEFING — Something Is Leaking!

Your team intercepted `classified.enc`, an image encrypted using **AES-256-ECB** and an OpenSSL password. The original contains **432 × 475 grayscale pixels**, stored as raw bytes (one byte per pixel, no image header). **Your objective:** Discover what the ciphertext reveals *without* knowing the password; then recover the image and improve its encryption.
Rule: Do not use your password from Mission 1 until Challenge 3.

Challenge 1 — Investigate the Evidence

When OpenSSL encrypts with a password and its traditional *salted format*, it derives cryptographic key material from the password and a random **salt**. The salt is public, not the secret key. The file contains a recognizable header followed by the salt and ciphertext. In its traditional salted format, OpenSSL uses an 8-byte signature followed by an 8-byte salt by default Your job is to identify the boundaries.

- Inspect the first 64 bytes of `classified.enc` using:

```
xxd -l 64 classified.enc
```

Note: xxd displays a file's raw bytes in hexadecimal format, allowing us to inspect its structure without decrypting it.

Question 1: Invetigate the output, identify the OpenSSL signature and salt, and deduce how many bytes must be excluded before analyzing the actual ciphertext.

Challenge 2 — The Encrypted Image Mystery

ECB encrypts each block independently: $C_i = E_K(P_i)$. Identical plaintext blocks therefore generate identical ciphertext blocks under the same key.

2.1 Write your own program `investigate` to examine the ciphertext **without decrypting it**. Exclude any non-image bytes, identify equal 16-byte blocks, and assign the same visible grayscale value to equal blocks. Save the visualization as `pattern_ecb.png`.

2.2 What can you recognize?

CHECKPOINT — Security Analyst's Finding

You've revealed a pattern, not the secret! **Now unlock the original image!**

STUCK? — Python Rescue Kit

Hint 1 — Check the image size
The original image is 432 × 475 pixels, with one byte per grayscale pixel.
Useful command:

```
wc -c < classified.enc
```

Remember to account for the OpenSSL header and AES padding.

Hint 2 — Look for repeated blocks
AES processes 16-byte blocks. Useful Python library: `collections.Counter`

Hint 3 — Reconstruct the pattern
You do not know the original pixel values, but you know which ciphertext blocks are identical. Assign the same grayscale shade to identical blocks and preserve their original positions.

Hint 4 — Create the image
Useful Python library: `PIL.Image` (Pillow).

Remember: You are revealing a pattern, not decrypting AES!

Challenge 3 — Unlock the Original

You have discovered a pattern, but the original image is still encrypted. It is time to use the password recovered in Mission 1!

OpenSSL provides the `enc` command for encryption and decryption. By default, it encrypts; adding `-d` switches to decryption.

To recover the file, you must use the same encryption algorithm and key-derivation method:

- `-aes-256-ecb`: AES-256 in ECB mode.
- `-pbkdf2`: derive the AES key from your password.
- `-p`: display the cryptographic parameters used.

Now unlock the file:

```
openssl enc -d -aes-256-ecb -pbkdf2 -p \
  -in classified.enc -out recovered.raw
```

Enter your password from Mission 1 when prompted.

One last step! The decrypted file contains raw pixel data, not a PNG image. Use Python and Pillow to reconstruct a 432 × 475 grayscale image and save it as `recovered.png`.

CHECKPOINT — Secret Unlocked!

Mission accomplished! You have recovered the original image using your RSA password.
What was hidden that the ECB pattern could not fully reveal?
But is the encryption secure? Time to fix ECB!

Challenge 4 — Repair with CBC

Your security team wants to stop this kind of visual leakage.

4.1 Encrypt `recovered.raw` using **AES-256-CBC**. Name the output `classified_cbc.enc`.

4.2 Modify your investigation program to analyze the CBC ciphertext. Can you still recognize the original picture? Compare repeated-block counts and explain the difference using CBC chaining.

— End of Mission 2 —

Mission 3 — RSA: Keys, and Encryption

MISSION BRIEFING — Become an RSA Analyst

You have already used RSA mathematics to recover a secret password. Now it is time to work with real RSA keys using OpenSSL!
Your mission is to generate keys, investigate weak RSA, and exchange encrypted messages.

Part A — Key Generation

OpenSSL can generate an RSA private key using:

```
openssl genrsa -out key.pem size
```

5.1. Generate a small RSA private key of **512 bits**, named `mySmall.pem`.

5.2. Display information about your private key.

Syntax: To obtain information on a key, we can use:

```
openssl rsa -in key.pem -text -noout
```

5.3. Examine the content of `mySmall.pem`. Is the private key encrypted? Is your key safely stored in file `mySmall.pem`.

3.4 Break the 50-bit RSA public key `otherSmall_pub.pem`: find its two prime factors p and q and compute the private exponent d.

```
-----BEGIN PUBLIC KEY-----
MCIwDQYJKoZIhvcNAQEBBQADEQAwDgIHAtcS9BRJHQIDAQAB
-----END PUBLIC KEY-----
```

Hint. Extract n and e with `openssl rsa -pubin -in otherSmall_pub.pem -noout -text`, then factor n with a short program (test the divisors up to $\sqrt{n}$).

Hint — Need help? Extract the modulus n and public exponent e, `openssl rsa -in otherSmall.pem -noout -modulus` then factorize n using a small program. Remember that $n = pq$.

5.4. Try to apply the same factorization approach to the modulus of your 512-bit key. Can you recover its prime factors in a reasonable time?

Protecting private keys

A private key can be protected using symmetric encryption. OpenSSL supports AES-256-CBC for encrypting private-key files.

5.6. Generate two RSA keys of **2048 bits**:

- `private1.pem`: without encryption.
- `private2.pem`: encrypted using AES-256-CBC.

5.7. Retrieve the information contained in both private keys.

5.8. Extract the public keys associated with `private1.pem` and `private2.pem`. Save them as `public1.pem` and `public2.pem`.

Syntax To retrieve the public key associated with a private key, we use:

```
openssl rsa -in private-key -pubout -out public-key
```

5.9. Should a public key be stored in plaintext?

5.10. Display information about both public keys.

Part B — Encryption and Decryption

RSA allows a sender to encrypt a short message using the recipient's public key.

Syntax OpenSSL uses `pkeyutl` for public-key encryption and decryption.

```
openssl pkeyutl -encrypt -pubin -inkey public-key -in plaintext -out cipher
```

The `-decrypt` option performs decryption using the corresponding private key.

5.11. Create a short text file. Encrypt it using an RSA public key, then decrypt it using the associated private key. Verify that the recovered text matches the original.

5.12. Try encrypting a large text file using RSA. What happens? Why is RSA unsuitable for directly encrypting large files?

5.13. Exchange public keys with a lab partner. Encrypt a short message using your partner's public key and send the ciphertext. Ask your partner to decrypt it.

5.14. RSA cannot efficiently encrypt large files.

Using what you learned about AES in Mission 2, implement a **hybrid encryption scheme**:

5.15. Ask your lab partner for their RSA public key. Encrypt a long message using AES-256-CBC with a randomly generated AES key and IV. Encrypt the AES key using your partner's RSA public key.

Send your partner the encrypted message, the encrypted AES key, and the IV. Ask them to recover the original message using their RSA private key.

Why is combining AES and RSA more suitable for encrypting large messages?